Diseño de Componentes de EyBrek

Integrantes: Josué Herrera Quintero, María José Herrera Bonilla

Grupo 3

Carmen Cecilia Dulcey L.

Programa de Ingeniería de Software, Universidad de Santander

26 de octubre de 2026

Diseño en el Nivel de Componentes del Sistema EyBrek

EyBrek es una aplicación de escritorio para Windows que bloquea la pantalla de forma periódica durante un tiempo configurable y muestra ejercicios visuales durante la pausa, con el fin de promover descansos regulares y cuidar la salud visual.

El sistema sigue una arquitectura cliente-servidor en tres capas: un cliente de escritorio en Electron, un servidor en Node.js con Express y una base de datos MySQL.

Este documento presenta el diseño a nivel de componentes, entendido como la agrupación de clases relacionadas en unidades modulares que ofrecen interfaces definidas y pueden reutilizarse en distintos contextos.

Componentes Principales y sus Responsabilidades

El sistema se organiza en ocho componentes, cuatro en el cliente y cuatro en el servidor

Tabla 1

Componentes de EyBrek, ubicación, responsabilidad e interfaz

Componente	Ubicación	Responsabilidad	Interfaz que ofrece
Interfaz de usuario	Cliente	Mostrar las pantallas de inicio de sesión, registro y configuración, y recibir lo que escribe el usuario.	Ninguna; consume ITemporizador e IComunicacion
Temporizador	Cliente	Decidir cuándo bloquear: cuenta el intervalo y verifica el horario y el estado del bloqueo.	ITemporizador
Bloqueo de pantalla	Cliente	Mostrar el bloqueo con la hora, la hora de fin y el ejercicio, y desbloquear al terminar.	IBloqueo
Comunicación y copia local	Cliente	Hablar con el servidor y guardar la copia local de la configuración.	IComunicacion
Autenticación	Servidor	Registrar usuarios, iniciar y cerrar sesión, y validar tokens.	IAutenticacion
Configuración	Servidor	Guardar, leer y validar las preferencias, y crear la configuración por defecto.	IConfiguracion

Ejercicios	Servidor	Entregar el siguiente ejercicio de la rotación y guardar el progreso.	IEjercicios
Acceso a datos	Servidor	Leer y escribir en MySQL; es el único que conoce las tablas.	IAccesoDatos

Componente Seleccionado: Bloqueo de Pantalla

Se seleccionó el componente Bloqueo de pantalla porque concentra el propósito central de EyBrek: bloquear la pantalla sin opción de omitirla y mostrar el ejercicio durante el descanso. Este componente recibe la duración y los ejercicios como datos, por lo que no conoce la red, el servidor ni la base de datos.

Clases que lo Conforman

El componente se compone de cinco clases, GestorBloqueo coordina, VentanaBloqueo dibuja, CuentaRegresiva cuenta y PanelEjercicios muestra los pasos, GestorBloqueo realiza la interfaz IBloqueo y compone a VentanaBloqueo y CuentaRegresiva; a su vez, VentanaBloqueo compone a PanelEjercicios, que muestra uno o más EjercicioVisual, cuando la cuenta regresiva llega a cero, GestorBloqueo cierra la ventana y emite el evento bloqueoTerminado.

Tabla 2

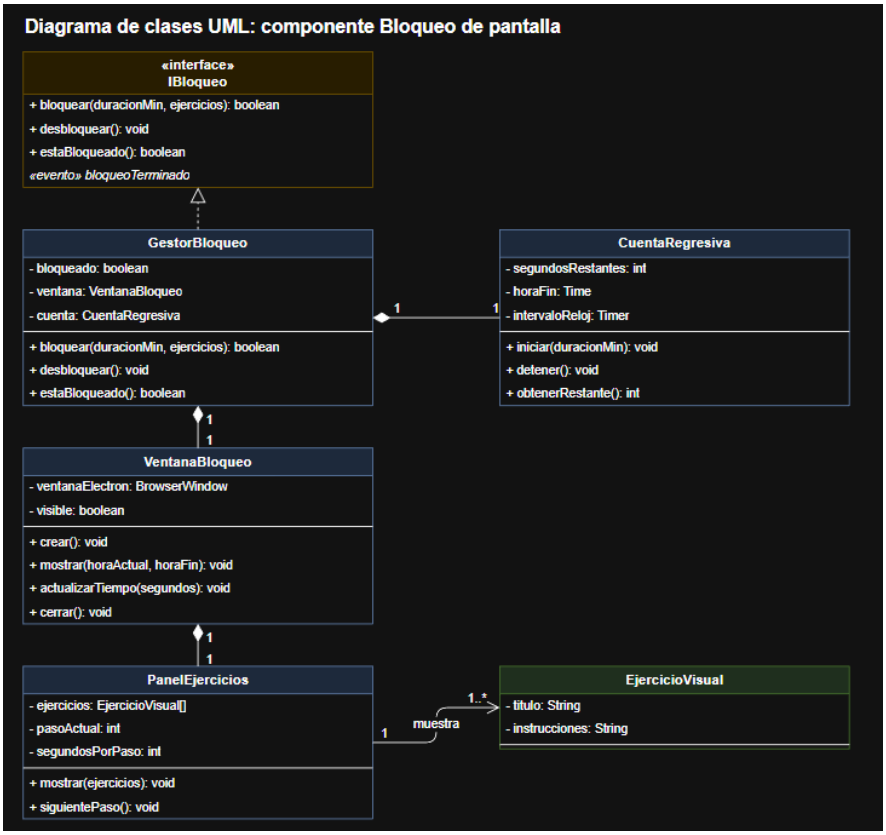
Clases del componente Bloqueo de pantalla

Clase	**Atributos**	**Métodos principales**
GestorBloqueo	- bloqueado: boolean - ventana: VentanaBloqueo - cuenta: CuentaRegresiva	+ bloquear(duracionMin, ejercicios): boolean + desbloquear(): void + estaBloqueado(): boolean
VentanaBloqueo	- ventanaElectron: BrowserWindow - visible: boolean	+ crear(): void + mostrar(horaActual, horaFin): void + actualizarTiempo(segundos): void + cerrar(): void
CuentaRegresiva	- segundosRestantes: int - horaFin: Time	+ iniciar(duracionMin): void + detener(): void

	- intervaloReloj: Timer	+ obtenerRestante(): int
PanelEjercicios	- ejercicios: EjercicioVisual[] - pasoActual: int - segundosPorPaso: int	+ mostrar(ejercicios): void + siguientePaso(): void
EjercicioVisual	- titulo: String - instrucciones: String	Sin métodos; transporta los datos del ejercicio.

Figura 1

Diagrama de Clases UML del Componente Bloqueo de Pantalla



Interfaces y Servicios

Los componentes se comunican únicamente a través de las interfaces; ninguno lee el interior de otro, para el componente seleccionado, el servicio que se ofrece es IBloqueo. Entre el cliente y el servidor la comunicación se realiza mediante una API REST con datos en formato JSON sobre HTTP, y entre el servidor y la base de datos mediante consultas SQL parametrizadas.

Tabla 3

Interfaces del sistema y sus operaciones

Interfaz	**Ofrecida por**	**Operaciones**
ITemporizador	Temporizador	iniciar(configuracion), detener(), reiniciar(); eventos avisoProximo y bloquear
IBloqueo	Bloqueo de pantalla	bloquear(duracionMin, ejercicios): boolean, desbloquear(): void, estaBloqueado(): boolean; evento bloqueoTerminado
IComunicacion	Comunicación y copia local	iniciarSesion(correo, contraseña), registrar(nombre, correo, contraseña), cerrarSesion(), obtenerConfiguracion(), guardarConfiguracion(config), obtenerSiguienteEjercicio()
IAutenticacion	Autenticación	registrar(...), iniciarSesion(...), cerrarSesion(token), validarToken(token)
IConfiguracion	Configuración	obtener(idUsuario), guardar(idUsuario, config), crearPorDefecto(idUsuario)
IEjercicios	Ejercicios	siguienteEjercicio(idUsuario): { titulo, instrucciones }
IAccesoDatos	Acceso a datos	leerUsuarioPorCorreo(...), guardarConfiguracion(...), leerEjercicioPorOrden(...), entre otras

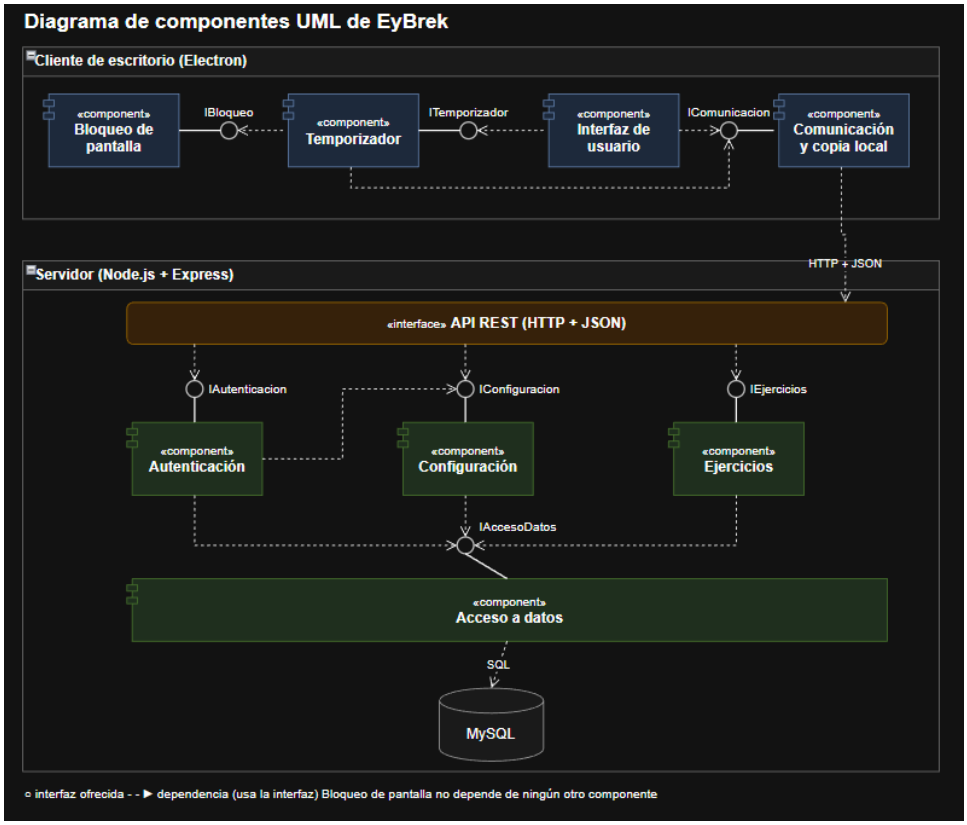
Diagrama UML de Componentes

Los componentes, las interfaces ofrecidas y las dependencias, en el cliente, la Interfaz de usuario usa ITemporizador e IComunicacion, y el Temporizador usa IBloqueo para bloquear la pantalla e IComunicacion para obtener la configuración. Bloqueo de pantalla no depende de ningún otro componente. Comunicación y copia local es el único componente del cliente que habla con el servidor, a través de la API REST.

En el servidor, la API REST entrega las peticiones a Autenticación, Configuración y Ejercicios. Autenticación usa IConfiguracion para crear la configuración por defecto al registrar un usuario, y los tres componentes usan IAccesoDatos, de modo que solo Acceso a datos se conecta a MySQL. Además, el Temporizador aplica el patrón Observer (Gamma et al., 1994): emite el evento avisoProximo, al que se suscribe la Interfaz de usuario, sin necesidad de conocer a quien lo escucha.

Figura 2

Diagrama de Componentes UML de EyBrek



Componente Reutilizable: Autenticación

Se eligió Autenticación porque cualquier sistema con cuentas necesita registrar usuarios, iniciar y cerrar sesión y validar tokens, y eso no depende de EyBrek: el componente no sabe nada de bloqueos, ejercicios ni temporizadores. Ofrece cuatro operaciones (registrar, iniciarSesion, cerrarSesion y validarToken) y solo toca las tablas Usuario y Sesion.

Compatibilidad

Su interfaz es REST con JSON, por lo que puede consumirla cualquier cliente, ya sea de escritorio, web o móvil. Trabaja sobre Node.js, Express y MySQL, la misma pila del servidor, y

no necesita librerías propias del cliente. Solo requiere IAccesoDatos, así que si otro proyecto usa otra base de datos, se cambia ese componente y Autenticación permanece igual.

Seguridad

Las contraseñas se almacenan con hash y nunca en texto plano, se propone un algoritmo con sal, como bcrypt, cada sesión se identifica con un token único y todas las consultas son parametrizadas, lo que previene la inyección SQL, como limitación, en red local la comunicación va por HTTP sin cifrar, por lo que se propone HTTPS como mejora futura.

Mantenimiento

Si cambia el algoritmo de hash o la forma de generar tokens, solo se modifica este componente, asimismo, el cambio futuro de recuperar la contraseña por correo queda contenido en Autenticación y Acceso a datos, más un servicio de correo nuevo, sin tocar Temporizador, Bloqueo de pantalla, Configuración ni Ejercicios, el componente tiene una sola razón para cambiar: la gestión de cuentas.

Dependencia con Configuración

Al registrar un usuario, Autenticación llama a crearPorDefecto() de Configuración para asignar los valores por defecto, se decidió mantener esta dependencia directa porque garantiza que el registro y la configuración inicial se completen juntos, sin dejar una cuenta sin configuración, para reutilizar el componente en otro proyecto basta con reemplazar esa única llamada, se consideró como alternativa emitir un evento usuarioRegistrado, pero se descartó por añadir un mecanismo extra y el riesgo de que la configuración no se cree si falla la suscripción.

Bajo Acoplamiento y Alta Cohesión

Alta Cohesión

Cada componente tiene una sola razón para cambiar. Autenticación gestiona cuentas y tokens; Configuración guarda y valida preferencias; Ejercicios administra la rotación y el progreso; el Temporizador decide cuándo bloquear; Bloqueo de pantalla muestra el bloqueo y desbloquea; Comunicación y copia local habla con el servidor y guarda la copia local; la Interfaz de usuario muestra pantallas y recibe datos; y Acceso a datos lee y escribe en MySQL. Dentro de Bloqueo de pantalla cada clase también hace una sola cosa, y la separación entre datos y presentación se mantiene porque la rotación vive en el servidor y el panel solo muestra lo que recibe.

Bajo Acoplamiento

Los componentes se comunican solo por interfaces y ninguno lee el interior de otro. El Temporizador aplica el patrón Observer, por lo que agregar una reacción nueva al aviso previo no obliga a modificarlo. Bloqueo de pantalla recibe la duración y los ejercicios como datos y no conoce la red, el servidor ni la base de datos. El SQL está en un solo lugar: si cambia la base de datos, solo se modifica Acceso a datos.

Los ejercicios se guardan como datos y no como código (RNF-07), de modo que agregar uno es insertar una fila. La dirección del servidor se define en un archivo de configuración del cliente, por lo que cambiar el servidor de equipo no requiere modificar código. Por último, las dependencias van en una sola dirección (cliente, API, componentes del servidor, Acceso a datos y MySQL) y no hay ciclos.

Evidencia y Limitación

Como evidencia, el cambio futuro de recuperar la contraseña (RF-25) afecta solo a Autenticación, Acceso a datos, la Interfaz de usuario y Comunicación. Como limitación, Autenticación depende de Configuración en el registro; es un acoplamiento aceptado y acotado a una sola llamada, crearPorDefecto().

Conclusiones

Dividir EyBrek en ocho componentes con responsabilidades claras permite cambiar una parte, como la base de datos, la regla de rotación o el diseño del bloqueo, sin tocar el resto. Las interfaces explícitas reducen el acoplamiento, y Autenticación resulta el componente más reutilizable porque no depende del dominio de EyBrek salvo por una llamada acotada. El diagrama de clases de Bloqueo de pantalla muestra que la alta cohesión también se aplica dentro de un componente.

Referencias

Dulcey, C. C. (2026). *Diseño en el nivel de componentes: Conceptos y ejemplos* [Diapositivas de clase]. Universidad de Santander.

Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Herrera Quintero, J. y Herrera Bonilla, M. J. (2026). *EyBrek: Proyecto de aula, avance del segundo corte* [Documento no publicado]. Universidad de Santander.

ISO/IEC/IEEE. (2018). *ISO/IEC/IEEE 29148:2018. Systems and software engineering - Life cycle processes - Requirements engineering*.